

ShadowNPU: System and Algorithm Co-design for NPU-Centric On-Device LLM Inference

Wangsong Yin
Key Lab of HCST (PKU), MOE; SCS,
Peking University, China
Beijing, China
yws@stu.pku.edu.cn

Daliang Xu
BUPT
Beijing, China
xudaliang@bupt.edu.cn

Mengwei Xu
BUPT
Beijing, China
mwx@bupt.edu.cn

Gang Huang
Key Lab of HCST (PKU), MOE; SCS,
Peking University, China
Beijing, China
hg@pku.edu.cn

Xuanzhe Liu*
Key Lab of HCST (PKU), MOE; SCS,
Peking University, China
Beijing, China
liuxuanzhe@pku.edu.cn

Abstract

Running LLMs on devices like smartphones has become a key catalyst towards privacy-preserving mobile AI. In state-of-the-art frameworks, the attention operator falls back from the dedicated NPU to the public CPU/GPU due to its sensitivity to quantization. Such a fallback leads to hampered user experience and extra system scheduling complexity. To realize NPU-centric LLM inference, this paper presents shadowNPU, a system-algorithm codesigned sparse attention module with minimal reliance on CPU/GPU by only sparsely computing a very small number of tokens. The key idea is to hide the overhead of estimating the important tokens by offloading it to NPU. On top of this, it further incorporates insightful techniques including NPU compute graph bucketing, head-wise NPU-CPU/GPU pipeline and per-head fine-grained sparsity ratio to achieve high accuracy and efficiency. Compared to design alternatives, shadowNPU achieves the best performance with strictly limited CPU/GPU resource; it requires much less CPU/GPU resource to achieve on-par performance of SoTA frameworks.

CCS Concepts

• **Computer systems organization** → *System on a chip*; • **Computing methodologies** → *Natural language generation*.

Keywords

NPU-Centric Inference, Sparse Attention on Mobile SoCs

ACM Reference Format:

Wangsong Yin, Daliang Xu, Mengwei Xu, Gang Huang, and Xuanzhe Liu. 2026. ShadowNPU: System and Algorithm Co-design for NPU-Centric On-Device LLM Inference. In *The 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys '26)*, June 21–25, 2026, Cambridge, United Kingdom. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3745756.3809205>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MobiSys '26, Cambridge, United Kingdom*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2027-7/2026/06
<https://doi.org/10.1145/3745756.3809205>

1 Introduction

On-device Large Language Models (LLMs) have been a critical enabler to privacy-preserving and ubiquitous artificial intelligence. Recently, the emergence of mobile-size LLMs like Qwen series [12, 46] catalyzes killer applications such as content summarization and GUI/API agents [7, 17, 65, 81].

NPU-centric LLM inference. Mobile devices typically incorporate heterogeneous processors such as CPU, GPU, and NPU. Ideally, the LLM inference shall be end-to-end executed on the mobile NPUs with minimal out-of-NPU resources. The advantages of such NPU-centric LLM inference are twofold. First, mobile NPUs, as dedicated for neural networks, are with high-throughput integer capability and much more power-saving than CPU/GPU [16, 50, 60, 67]. Second, it avoids resource contention with other general-purpose mobile workloads (e.g., user interaction handling and rendering) that rely on CPU and GPU. Co-locating LLM inference with those workloads can easily hamper user experience, adding extra complexity to the system scheduling [19, 24, 55, 74, 86].

However, in practice we find the attention operation in LLMs often falls back to mobile CPU/GPU in state-of-the-art on-device inference frameworks [16, 21, 39, 67]. For instance, *llm.npu* [67] offloads the non-attention operations to NPUs; yet the attention still runs on CPUs — not NPU-centric.

Such a compromise is underpinned by the rationale of quantization sensitivity. On one hand, activations in LLMs feature much more hard-to-quantize outliers compared to weights [31, 62], while the attention operation computes on multiple activation tensors, i.e., the Q, K, V tensors. On the other hand, the NPU's static graph hampers fine-grained quantization. It sets superparameters (tensor shape and constants such as scale factor) before compilation for resource scheduling. Such a paradigm leads to sharing a fixed scale factor for an entire tensor, further limiting the attention quantization. In the following experiments (Table 3), we find that NPU-based attention leads to on average 18 pp accuracy drop on mobile LLMs/tasks. This echoes the necessity of offloading the attention to CPU/GPU.

NPU-centric LLM inference by sparse attention. A key opportunity to minimize the reliance on CPU/GPU resource is the the highly sparse feature of attention operations. A small portion of tokens can be much more important than others; sparsely computing

these tokens can significantly reduce the CPU/GPU computation with almost no accuracy drop. As detailed in §2, in the Qwen2-1.5B model, on average more than 80% of the tokens are assigned relatively low importance in the WikiText-2 dataset.

Surprisingly, despite the promised advantages, our preliminary results indicate that directly applying sparse attention in LLM inference yields no performance gain, as before the sparse computation can be executed, an estimation stage is required to evaluate token importance, incurring substantial overhead on the CPU/GPU. This stage involves a matrix multiplication to compute attention scores for each token in Q and K [59, 85]. Consequently, the estimation stage dominates the overall attention computation under high sparsity. For instance, with a sparsity ratio of 20% (i.e., 80% of tokens pruned), estimation alone contributes over 60% of the total overhead, reducing CPU/GPU resource consumption by only about 20%. Intuitively, we can downsample the Q and K for an efficient estimation, yet leading to significant accuracy drop. For instance, when eliminating the estimation overhead by grouping adjacent tokens in a block and only computing the block-level importance, the accuracy undergoes an average drop of 4 pp on mobile LLMs/tasks compared to token level sparsity. To this end, the key question of shadowNPU's design is how to compute sparse attention accurately and efficiently in NPU-centric LLM inference.

shadowNPU: Dynamic sparse attention with NPU-based estimation. In this paper, we propose shadowNPU, a system-algorithm co-designed on-device sparse attention module for NPU-centric LLM inference while minimizing its reliance on CPU/GPU fallback (e.g., only one CPU core). Its key idea is that the estimation of important tokens can be much more resilient to quantization compared to the end-to-end attention. The rationale is that determining the important tokens only requires the relative value of attention scores, while calculating the attention's exact result requires the absolute value. Thus, shadowNPU offloads the estimation to NPU, and transfers the position indices of the important tokens to CPU/GPU for further sparse attention calculation. By doing so, only a small portion of tokens are computed on CPU/GPU with high precision float operations. Besides, shadowNPU also identifies the uneven property of each head's sparsity ratio. It determines the ratio for each head by a lightweight offline profiling, further optimizing the accuracy-efficiency tradeoff. When designing shadowNPU, however, we need further to address the following major challenges that have not been explored in the existing literature.

(1) *The inflexibility of NPU static compute graph.* The software stack of NPUs employs a static compute graph, which specifies the shape and quantization scale factors as constant for offline optimization. However, the attention input tensors are highly dynamic; their scale factors exhibit large fluctuation. The tensor shape may also vary in different cases. Directly generating one static compute graph for the attention leads to accuracy drop and under utilization of NPU.

(2) *Multi-faceted NP-hard NPU-CPU/GPU scheduling.* The two stages of shadowNPU's sparse attention can be overlapped by a head-level fine-grained pipeline on both the NPU and CPU/GPU. However, achieving efficient fine-grained scheduling is non-trivial and challenging because (i) the pipeline can be optimized from various deeply buried aspects like NPU kernel fused launching. Its

performance may be degraded without carefully going through all of these aspects. (ii) the pipeline planning problem is NP-hard.

shadowNPU incorporates the following techniques with deep insights towards the aforementioned challenges.

(1) *NPU compute graph bucketing* (§3.3). To mitigate the limitations of static compute graph, shadowNPU offline generates multiple graphs and organizes them in buckets for online selection. Specifically, a bucket contains graphs with various shapes and the same scale factor of input tensors. At online inference stage, the bucket whose scaling factor is closest to that of the input tensor is selected, followed by selecting a graph with an appropriate shape from the bucket.

(2) *Head-wise NPU-CPU/GPU pipeline* (§3.4). shadowNPU proposes a head-level pipeline that leverages the following insights. Firstly, it overlaps the NPU estimation, CPU/GPU top k operation and CPU/GPU sparse attention with each other. On top of this, it launches the NPU kernels with the same scale in one-shot to maximize the NPU utilization, and carefully plans the execution order of each head to minimize the pipeline bubbles. To achieve fast on-device planning, it simplifies the NP-hard planning complexity by a heuristic greedy search, which selects the head that minimizes the pipeline latency at each step.

Implementation and Evaluation. We prototype shadowNPU in over 10,000 LoC of C++/Python. We test shadowNPU on commercial-off-the-shelf smartphones MI14 and Redmi K60 Champion Edition with snapdragon 8gen3/8gen2 equipped. We only use one CPU core for necessary control flow and sparse compute, with other compute completely on NPUs.

We test both the breakdown performance of the attention module and the end-to-end performance of integrating shadowNPU into the state-of-the-art NPU framework llm.npu [67]. The evaluation is conducted on mobile LLMs, including Qwen2-0.5B/1.5B and PhoneLM-0.5B/1.5B. We report the results on three representative datasets of mobile-specific tasks (NLP and agents). Compared to four design alternatives with a circumstance of highly limited CPU/GPU resources, shadowNPU exhibits up to 6.9× breakdown speedup, up to 4.5× end-to-end speedup and up to 7.7× energy reduction. Compared to native attention of SoTA frameworks, shadowNPU achieves on-par or even better performance with significantly fewer CPU/GPU resources. The accuracy loss is only 0.4 pp on average on four models and three datasets.

Contributions are as follows.

- We identify the significant problem of NPU-centric LLM inference on mobile devices, and propose shadowNPU, a system-algorithm codesigned sparse attention that significantly reduces the reliance on float compute of CPU/GPU.
- We propose several key techniques, including NPU offloading of token importance estimation, NPU compute graph bucketing and NPU-CPU/GPU pipeline, and tackle the main challenges of NPU-centric on-device attention.
- shadowNPU outperforms strong baselines while minimizing CPU/GPU resource. shadowNPU can be plug-and-play to any mainstream on-device LLM inference frameworks.

Vendor	SoC	NPU	INT Capability	Float Capability
Qualcomm	Snapdragon	Hexagon DSP	High	No
Qualcomm	Snapdragon	HTP	High	Low
Texas Instruments	TMS320F2812	C28x DSP	High	No
Tesla	FSD	FSD D1	High	No

Table 1: NPUs of mainstream mobile/embedded SoCs. High-throughput integer capability is the main focus of NPUs.

Framework	llm.npu	HeteroLLM	llama.cpp	mlc-llm
Atten.	CPU	GPU	CPU	GPU
Non-Atten.	NPU	GPU/NPU	CPU	GPU

Table 2: On-device frameworks typically run the attention operation on CPU/GPUs.

2 Background and Motivation

2.1 LLM Inference on Mobile SoCs

Mobile NPUs: integer accelerators for DNN inference. Due to increasing concerns about individual data protection and safety, more users are opting to run large language models directly on their personal devices rather than sending sensitive information to remote servers. To achieve higher performance, on-device LLM inference shall be executed on NPUs, specialized hardware designed for DNN inference. Such paradigm is referred as *NPU-centric inference*. Table 2 summarizes the NPUs on mainstream mobile/embedded SoCs, including Qualcomm Snapdragon series [6, 8], Texas Instruments TMS320F2812 [9], etc.

We observe that the commonality among these NPUs is their primary focus on *low-precision integer compute*, i.e., all of them support integer operation, but have zero or relatively low float capabilities. This situation arises because (i) DNNs are able to be quantized to lower precisions for efficient execution and memory saving compared to general workloads. (ii) supporting multiple data precision types consumes more chip area or introduces significant additional design complexity for unit reusing [33, 34, 42].

The static compute graph of mobile NPUs. To schedule the resources for the best runtime efficiency, mobile NPUs typically employ a static compute graph. Figure 1 illustrates the workflow of a static-graph-based inference procedure for QNN SDK [5]. In the offline stage, a computation graph is first built and then loaded into the device memory. During the online inference stage, the input data is fed into the graph for execution. Finally, the graph is offline released from the memory.

The static graph requires several key hyperparameters to be specified and fixed during the offline graph-building stage. As illustrated in Figure 1, this procedure includes assigning a single float value as the scale factor for an INT8 tensor and defining the tensor exact shape. The tensors and an operator type provided by the vendor are then assembled into a compute graph. Finally, the graph is compiled into a binary file through a series of optimization passes.

The time overhead of each stage is listed in Figure 1. The offline stage is extremely time-consuming compared to the online stage.

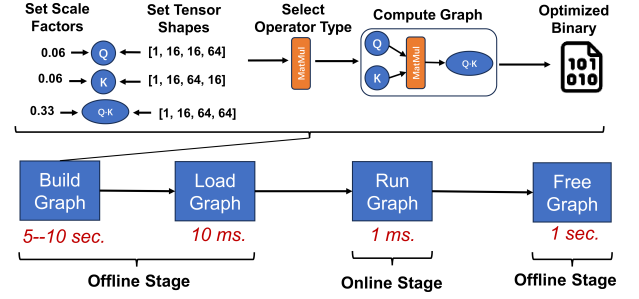


Figure 1: The workflow of static compute graph of mobile NPUs. The latency is acquired on QNN SDK [5] by a basic matrix multiplication operation.

Dataset	PhoneLM -0.5B		PhoneLM -1.5B		Qwen2 -0.5B		Qwen2 -1.5B	
	C/G	N	C/G	N	C/G	N	C/G	N
ArxivSum [18]	14.7	0.0	11.9	0.0	10.7	9.4	8.5	9.1
DroidCall [65]	27.5	20.5	20.5	19.0	34.5	27.5	48.0	22.5
Octopus [17]	64.6	24.1	79.2	24.7	60.6	34.8	61.2	34.2

Table 3: Accuracy on mobile LLMs and tasks. “C/G” means running attention on CPU/GPU in float32; “N” means running attention on NPU in INT8.

This makes it impossible to dynamically online build the graph for each input.

The attention falls back to CPU/GPUs in state-of-the-art on-device LLM inference frameworks. Table 2 lists representative on-device LLM inference frameworks, including llm.npu [67], HeteroLLM [16], llama.cpp [21] and mlc-llm [39]. They all delegate the attention operation to CPU or GPU. For instance, although llm.npu offloads its non-attention parts to NPU, the attention operation still runs on CPU; HeteroLLM also employs such an NPU-CPU/GPU collaborated paradigm. The CPU-based framework llama.cpp and GPU-based framework mlc-llm also show this property. This is mainly because that the attention is unfriendly to mobile NPUs. The attention compute involves multiple activations (i.e., the non-weight tensors in a neuron network), which are hard to quantize [31, 62]. Further, the static graph limits the quantization on NPUs to per-tensor static quantization. Such a quantization method is very coarse-grained. It shares a fixed scale factor across an entire tensor, preventing the NPU from adopting fine-grained methods such as K-quant [21] or AWQ [31]. Table 3 shows that offloading the attention operation to mobile NPUs leads to significant accuracy degradation. For instance, PhoneLM-0.5B shows a 14.7 pp accuracy drop and a 7 pp accuracy drop on ArxivSum dataset and DroidCall dataset, respectively. On three datasets and four models, the NPU-based attention exhibits an average drop of 18 pp.

The reliance on CPU/GPU fallback for high accuracy makes the on-device inference not NPU-centric. It fails to leverage the high-throughput compute capability and low energy consumption of mobile NPUs. Even worse, it leads to severe resource contention between other mobile apps co-located on CPU/GPU.

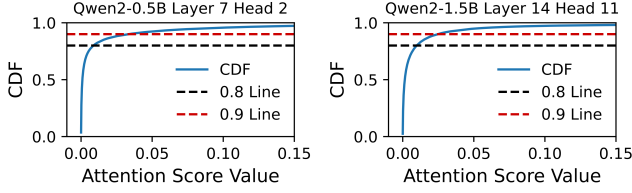


Figure 2: The attention score skewness of LLMs. Profiled on 128 samples from WikiText-2.

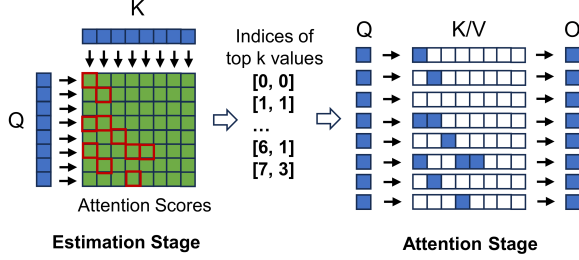


Figure 3: The workflow of sparse attention.

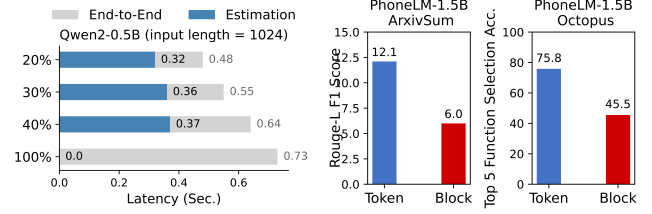
2.2 Sparse Attention

The opportunity of minimizing the reliance on CPU/GPU is the highly sparse characteristic of attention operation.

The attention can be highly sparse. We observe that only a small fraction of tokens in the attention mechanism are truly important. We evaluate 128 randomly sampled data points from the WikiText-2 corpus [37], analyzing two randomly selected attention heads from Qwen2-0.5B and Qwen2-1.5B. The results are shown in Figure 2. For Qwen2-0.5B, more than 80% attention scores fall below 0.01; over 90% attention scores fall below 0.03. A similar trend is observed in Qwen2-1.5B. These near-zero values contribute negligibly to the attention output and, therefore, need not be computed. In other words, the reliance of attention on CPU/GPU computation can be substantially reduced.

The workflow of sparse attention. Figure 3 shows the general workflow of a sparse attention. This workflow is flexible enough to encompass various empirical sparsity patterns—such as sliding windows or vertical-slash patterns [25–27], and can also handle sparsity without obvious structural patterns. It consists of two stages: (1) *Estimation stage*. An attention score matrix is calculated by $\text{attentionScores} = \text{softmax}(\text{mask}(\frac{Q \cdot K}{\sqrt{d_k}}))$, with all tokens in Q and K involved. After that, top k values out of the attention scores are selected, with their indices transferred to the next stage. (2) *Attention stage*. Only the tokens that are retained by the indices from the estimation stage is involved in the sparse compute of $O = \text{softmax}(\text{mask}(\frac{Q \cdot K}{\sqrt{d_k}})) \cdot V$.

Bottleneck: the estimation stage. Despite the potential advantages of sparse attention, directly applying it in LLM inference provides little performance benefit. As shown in Figure 4a, given a sparsity ratio of 40%/30%/20%, the end-to-end latency of an attention kernel only speeds up by 12.4%/24.7%/34.3%. This is because, unlike the attention stage that computes only the important tokens,



(a) Latency breakdown.

(b) Accuracy drop.

Figure 4: (a): The breakdown latency of an attention kernel that runs on a CPU mid core of snapdragon 8egn3 SoC. (b): The accuracy drop when grouping 64 tokens as a block for estimation on mobile LLMs and tasks.

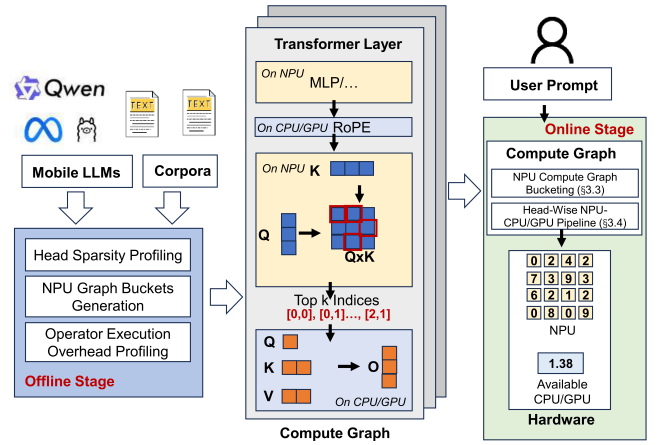


Figure 5: The workflow of shadowNPU.

the estimation stage requires processing all tokens, dominating the overall computation when the attention is extremely sparse. For instance, in a kernel with 20% sparsity (80% tokens are discarded), 66.7% out of the end-to-end computation is attributed to the estimation. An intuitive approach is to group the adjacent tokens in Q and K into a block before the estimation, and use the attention score of the entire block to represent the importance of each token in this block, i.e., block sparse attention [73], reducing the estimation overhead. However, the block sparse attention degrades the accuracy of mobile LLMs significantly, as it overlooks some important tokens due to its coarse-grained estimation. For instance, in Figure 4b, PhoneLM-1.5B shows a 6.1 pp accuracy drop on ArxivSum dataset. *In a nut shell, shadowNPU must carefully design the estimation stage to ensure high accuracy while keeping latency low.*

3 The Design

3.1 Overview

Design Goal. shadowNPU is a system–algorithm co-designed, NPU-centric LLM inference framework tailored for mobile SoCs. Its goal is to execute attention both accurately and efficiently, while

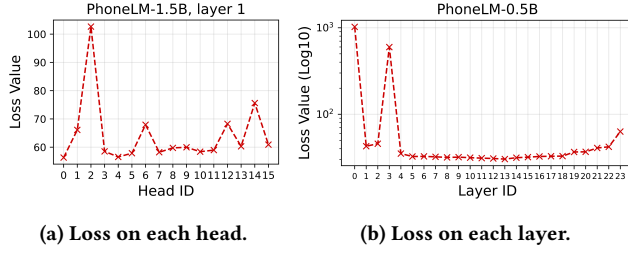


Figure 6: The importance is uneven across heads and layers.(a): Removing the heads in layer 1 of PhoneLM-0.5B; (b) removing the layers of PhoneLM-0.5B. The data is on 128 samples of WikiText-2. Loss values over $1e-3$ are clamped to $1e-3$. The y-axis of subfigure (b) is processed by log10.

minimizing reliance on general-purpose processors such as CPUs and GPUs.

Workflow of shadowNPU. Figure 5 shows the workflow of shadowNPU that consists the following parts.

- *The offline stage.* shadowNPU performs several kinds of lightweight profiling on general corpora (by default 128 samples from WikiText-2). Firstly, shadowNPU determines a sparse ratio for each separated head based on its importance and global sparsity ratio (configured by system). Then, shadowNPU generates multiple static graphs and organizes them in buckets for online inference. Besides, it also profiles the on-device execution overhead of key operators on the SoC with OS-configured available CPU/GPU resource for pipeline planning. This stage is mainly performed on cloud servers before installing the model to mobile devices, with negligible time and resource overhead (e.g., < 1 cloud GPU hours for each model).
- *The NPU-centric compute graph.* shadowNPU generates a sparse attention module. This module can be further integrated into the on-device inference frameworks as an end-to-end NPU-centric compute graph. As shown in Figure 5, the operators excluding the attention run on NPU by the inference framework. For the attention module, the RoPE is run on CPU/GPU, and the estimation of Q·K is run on NPU in INT8. After getting the top k positions, the CPU/GPU runs the fragmented QKV sparsely to get the output O . The compute and data movement outside the NPU is minimized.
- *The online stage.* The compute graph takes the user’s prompt text as the input, and executes each part on the CPU/GPU/NPU of mobile SoC. During inference, two optimizations are further introduced, i.e., NPU compute graph bucketing (§3.3) and Head-wise NPU-CPU/GPU pipeline (§3.4).

3.2 Dynamic Sparse Attention of shadowNPU

Head-specific sparse ratio. One of shadowNPU’s insight is that the sparse ratio of attention should be head-specific.

According to the progress of eXplainable AI (XAI) [38], the importance of a certain module of a neuron network can be measured by the delta loss after removing it. For instance, on a calibration dataset C , the importance of head i in layer j can be measured as

$$headImp_i = loss(head_i = 0, C) - loss(C), \quad (1)$$

Models	Various global sparse ratios				
	20%	30%	40%	50%	80%
PhoneLM-0.5B	99.32%	99.39%	99.45%	99.51%	99.79%
PhoneLM-1.5B	99.24%	99.27%	99.29%	99.34%	99.61%
Qwen2-0.5B	99.03%	99.32%	99.51%	99.59%	99.86%
Qwen2-1.5B	99.68%	99.80%	99.83%	99.84%	99.95%

Table 4: Predicting the important positions in the Q·K via NPU. Data: 128 samples from WikiText-2. We report the average recall rate of all heads with the float QxK as ground truth. The rate is surprisingly high under various global sparse ratios.

and the layer j where the head is located can be measured by

$$layerImp_j = loss(layer_j = 0, C) - loss(C). \quad (2)$$

In Figure 6, we show the importance of head i and layer j . We remove the heads in the first layer in Figure 6(a), and show the loss after removal. The results exhibit obvious unevenness. For instance, the head 2 makes the loss value increase to over 100, yet the head 8 only makes the loss value increase to less than 60. Such an observation can also be found in Figure 6(b), which depicts the layer-level loss. The rationale behind the unevenness is mainly that various parts of a neuron network may learn various aspects of the data [15, 20, 26], especially in compact mobile LLMs.

In summary, for different heads in different layers, the sparsity ratio should also be uneven. An important head should retain more tokens, while a trivial head should retain less. Given a global average sparsity ratio r and the number N of all heads in a model, the sparsity ratio of head i is

$$\frac{r \cdot N \cdot clamp(headImp_i \cdot layerImp_j)}{sum(clamp(headImp_i \cdot layerImp_j))}, 1 \leq i \leq N, \quad (3)$$

where $clamp(\cdot)$ means clamping extremely large points (e.g., over $1e-3$) and $j = i/headNumPerLayer$. In doing so, the intrinsic feature of attention heads is fully explored.

The sparse ratio of each head is determined at offline stage and stays fixed during online stage, incurring no overhead for inference. The default calibration dataset is 128 samples from WikiText-2. The offline profiling only takes about 5 minutes for a mobile LLM on a cloud server with a single A100 GPU, being affordable for most developers.

Running estimation on NPU. Another key insight of shadowNPU is that the estimation can be offloaded to low-precision NPU. shadowNPU’s observation is that only determining the important positions in Q·K is less prone to quantization compared to calculating the exact value of the QKV result. For instance, Table 4 shows the average recall rate of predicting the important positions in the Q·K via NPU’s INT8 per-tensor quantization. Surprisingly, the rate is more than 99% for various models and global sparsity ratios. This stands in stark contrast to the significant accuracy drop observed (e.g., 40.5% on Octopus, PhoneLM-0.5B) when using NPU to compute QKV. The rationale behind it is not hard to comprehend. The computation of QKV results requires precise “value equality”, whereas identifying the positions of larger elements in Q·K allows the computational results to fluctuate within a certain range. For example, the latter accepts both a Q·K = {0.4, 0.3, 0.2, 0.1} and a quantized Q·K = {0.5,

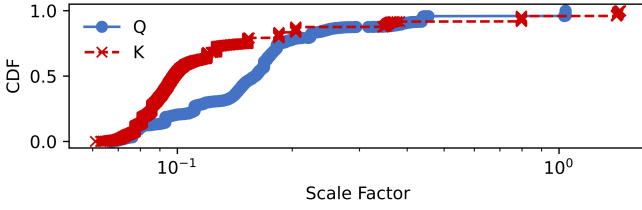


Figure 7: The CDF of each head’s scale factors of Q/K. Model: Qwen2-0.5B; data: 128 samples from WikiText-2. The x axis is logged by 10.

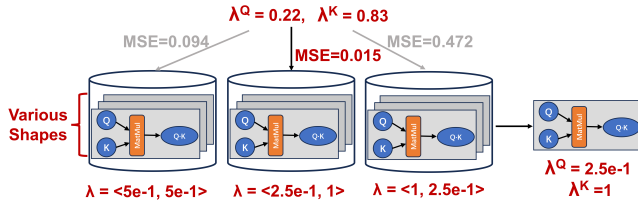


Figure 8: Bucketing the graphs to handle varying inputs.

0.4, 0.08, 0.02}, while the former only accepts the non-quantized Q-K.

On top of this, shadowNPU’s workflow of estimation stage is as follows. Firstly, it quantizes the Q and K tensor and runs dense INT8 Q-K on NPU. Then, the Q-K result is transferred to CPU/GPU to find the top k values. Finally, the CPU/GPU computes the sparse QKV according to the positions of the top k values. A detail here is that shadowNPU does not further compute the masked attention score based on Q-K. Since the softmax operation is strictly monotonically increasing, we can directly use the Q-K that before softmax for top k. Also, the causal mask is not applied to Q-K. Instead shadowNPU straightforwardly skips the masked positions for the top k operation. The skip is very convenient on CPU/GPU.

3.3 NPU Compute Graph Bucketing

The fluctuation of scale factors. As shown in Figure 7, we show the scale factors of each head in Q tensor and K tensor for NPU-based estimation stage, with a per-tensor (per-head) linear quantization of INT8. The data samples are the same to the aforementioned. The results show that the value of the scale factor varies significantly for each head in both Q and K. For instance, over 30% scale factors in Q tensor are larger than 0.2; in comparison, 20% scale factors are smaller than 0.1. Recall that to maximize the runtime efficiency, the software stack of NPU requires offline compiling a static compute graph that with fixed constant and shape. If we apply a single graph to all inputs, the accuracy will drop significantly since there is no one-size-fits-all scale factor as the graph constant. **Our solution: bucketing the graphs.** shadowNPU offline generates multiple graphs and cache them in buckets for online usage. Specifically, as shown in Figure 15, we denote λ^Q/λ^K as the scale factor of Q/K, and $\lambda = \langle \lambda^Q, \lambda^K \rangle$ as the constant of a QxK compute graph.

At offline stage, we profile the average $\bar{\lambda}$ on the calibration corpora, and generate $\{\langle \bar{\lambda}^Q, \bar{\lambda}^K \rangle, \langle \bar{\lambda}^Q \cdot \sigma, \bar{\lambda}^K / \sigma \rangle, \dots, \langle \bar{\lambda}^Q \cdot \sigma, \bar{\lambda}^K \cdot \sigma \rangle\}$ buckets, where σ is the step size (by default 5e-1, detailed later in §5.2). Notably, a bucket contains multiple graphs with various shapes¹ due to the NPU kernel fused launch (detailed in §3.4). The memory overhead of caching the graphs is negligible. Since a QxK graph does not contain any weight, a graph only takes about 10–100 KB.

At online stage, in Figure 15, given a pair of input tensors Q and K, we first calculate the MSE of $\langle \lambda^Q, \lambda^K \rangle$ to each bucket’s λ . Then, the input falls into the bucket with the smallest MSE. Finally, a compute graph with the corresponding shape is selected out for the following inference. In doing so, the accuracy of NPU estimation is guaranteed. The configuration details of bucketing the graphs are discussed in §5.

3.4 Head-Wise NPU-CPU/GPU Pipeline

Consider the execution of shadowNPU’s dynamic sparse attention. Figure 9 shows a minimal illustration with two heads. In each head, we need to first estimate the attention scores (i.e., the blue section), then select top k positions (i.e., the orange section), and finally do sparse QKV operations. Formally, if we use ζ_{npu}^i , ζ_{topk}^i , and ζ_{qkv}^i to represent executing the corresponding section of the i_{th} of n heads, the execution obeys

$$\zeta_{npu}^i \leftarrow \zeta_{topk}^i; \zeta_{npu}^i \zeta_{topk}^i \leftarrow \zeta_{qkv}^i, \forall i \in n, \quad (4)$$

where “ $\leftarrow$ ” means the dependency.

The naive way is running each operation sequentially (Figure 9(1)). However, this ignores several key optimizations in this procedure. shadowNPU further introduces the following insights.

Overlapping. Both ζ_{npu}^i and ζ_{qkv}^i operations are compute-bound, and the ζ_{topk}^i is relatively memory-bound. Furthermore, ζ_{npu}^i and ζ_{qkv}^i run on separate processors (NPU and CPU/GPU, respectively). Thus, according to Formula 4, if $\zeta_{npu}^i \zeta_{topk}^j$ and ζ_{qkv}^k have no dependency, then they can be executed in parallel. As shown in Figure 9(2), after overlapping these operations by a three-stage pipeline, the end-to-end latency can be reduced.

NPU kernel fused launch. The NPU is designed for high-throughput dense operations. A vanilla method of launching a single head for execution causes under-utilization of the IDU. For instance, the QxK compute graph of $Q=[1,1,128,64]$ and $K=[1,1,2048,64]$ (laid out in BHSD) takes 2ms on MI14; a graph that fuses 2 heads or 4 heads together only takes 3 ms or 4ms, respectively. On the other hand, since the number of scale factor buckets is finite (in fact a few buckets are enough, as detailed in §5.2), there exist many heads share the same scale factor. Thus, such heads can be fused together in a launch. As shown in Figure 9(3), head 1 and head 2 have the same scale factor. After fusing, the pipeline latency is further reduced.

Reordered execution. There exist bubbles in the pipeline due to the dependency in Formula 4. Since each head has its own sparsity ratio, the execution order of the heads influences the bubbles. A

¹The “shape” here mainly refers to the head dimension. On mobile devices, there is almost no batched input, and the sequence dimension is also fixed thanks to the chunked inference feature in modern on-device Transformer inference frameworks [67].

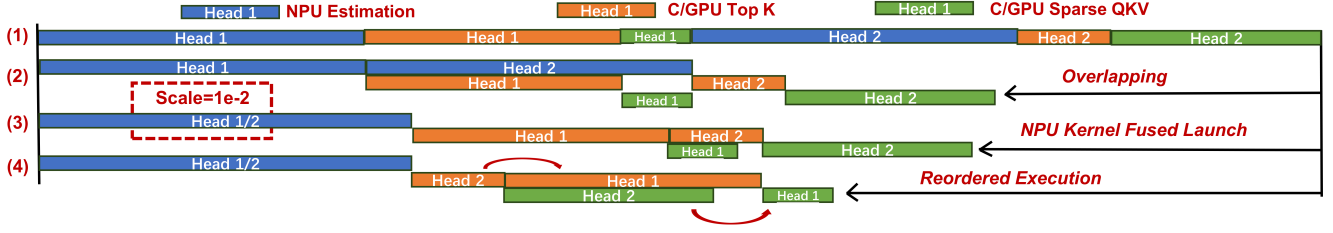


Figure 9: An illustration of NPU-CPU/GPU pipeline.

Algorithm 1 The pipeline planning policy.

Input: $heads_{c/gpu}$: each head of the attention. **Output:** res_{npu} , $res_{c/gpu}$: the execution order. t_{qkv} : the total execution time.

```

1:  $C/GPUPlan(t_{npu}, t_{topk}, t_{qkv}, fusedHeads)$ 
2:  $res = []$ 
3: for  $head$  in  $fusedHeads$ 
4:    $t_{min} = INF$ ,  $selectedHead = -1$ 
5:   for  $head$  in  $fusedHeads$ 
6:     if  $head$  in  $res$  then continue
7:      $t_{topk}^{new} = \max(t_{npu}, t_{topk}) + \text{topk time of head}$ 
8:      $t_{qkv}^{new} = \max(t_{qkv}, t_{topk}^{new}) + \text{qkv time of head}$ 
9:     if  $t_{qkv}^{new} < t_{min}$  then  $t_{min} = t_{qkv}^{new}$ ,  $selectedHead = head$ 
10:     $res.append(selectedHead)$ 
11:     $t_{topk} = \max(t_{npu}, t_{topk}) + \text{topk time of selectedHead}$ 
12:     $t_{qkv} = \max(t_{qkv}, t_{topk}) + \text{qkv time of selectedHead}$ 
13:  return  $res, t_{topk}, t_{qkv}$ 
14:  $Plan(heads_{c/gpu})$ 
15:  $heads_{npu} \leftarrow heads_{c/gpu}$  ▷ fuse the heads
16:  $t_{npu} = 0, t_{topk} = 0, t_{qkv} = 0, res_{npu} = [], res_{c/gpu} = []$ 
17: for  $fusedHeads$  in  $heads_{npu}$  ▷ a planning step
18:    $t_{min} = INF$ ,  $selectedHeads = None$ ,  $res = []$ 
19:   for  $fusedHeads$  in  $heads_{npu}$  ▷ greedy search
20:     if  $fusedHeads$  in  $res_{npu}$  then continue
21:      $t_{npu}^{new} = t_{npu} + \text{npu time of fusedHeads}$ 
22:      $res, t_{topk}^{new}, t_{qkv}^{new} = C/GPUPlan(t_{npu}, t_{topk}, t_{qkv}, fusedHeads)$ 
23:     if  $t_{qkv}^{new} < t_{min}$  then
24:        $t_{min} = t_{qkv}^{new}$ ,  $selectedHeads = fusedHeads$ 
25:    $res_{npu}.append(selectedHeads)$ ,  $res_{fsu} += res$ 
26:    $t_{npu} = t_{npu} + \text{npu time of selectedHeads}$ 
27:    $t_{topk}, t_{qkv} = C/GPUPlan(t_{npu}, t_{topk}, t_{qkv}, selectedHeads)$ 
28: return  $res_{npu}, res_{c/gpu}, t_{qkv}$ 

```

good order can squeeze out more bubbles. As shown in Figure 9(4), executing head 2 before head 1 further achieves latency reduction. **The pipeline planning policy.** shadowNPU’s pipeline puts the above optimizations together. Yet, solving the planning of such a pipeline has $O(n!)$ time complexity, which is NP-hard and unacceptable on mobile devices. To this end, shadowNPU employs a greedy search with polynomial complexity in Algorithm 1. Basically, we first do fused launch at the beginning of planning (line 15). Then, for each planning step, we select the ζ_{npu}^i , ζ_{topk}^i and ζ_{qkv}^i that minimize the current pipeline latency. The overhead of ζ_{npu}^i , ζ_{topk}^i and ζ_{qkv}^i is obtained in the lightweight offline stage mentioned in §3.1. Such a pipeline planning policy incurs negligible online overhead on mobile devices (e.g. < 1 ms on MI14).

4 Implementation

We prototype shadowNPU with 10,000 LoC in C++/python. We choose COTS smartphones with Qualcomm SoCs as the testbed,

Model	Q Heads	KV Heads	Dims	Layers
PhoneLM-0.5B [44]	16	16	64	24
PhoneLM-1.5B [45]	16	16	160	19
Qwen2-0.5B [47]	14	2	64	24
Qwen2-1.5B [48]	12	2	128	28

Table 5: Models we use in our experiments.

and shadowNPU can also work on other mainstream mobile devices. We only apply a middle core of ARM CPU as the minimal available CPU/GPU resource. We vary the available resource in the following experiments in §5.2. The NPU’s software stack is built upon QNN [5] and Hexagon SDK [2] of Snapdragon NPU. The CPU/GPU’s software stack consists of ARM NEON [1], OpenCL [4] and LLVM ecosystem [3], preserving the most runtime flexibility like dynamic graph execution. We fuse several operations, such as RoPE [53], quantization, matmul and softmax together in the kernel to minimize the redundant memory copy. Some non-sensitive instructions like multiplication inside them are further executed in half precision.

shadowNPU is compatible with any on-device LLM inference library. It can be integrated by directly including a header file, being least intrusive to the original codebase. In the following end-to-end experiments, we choose a high performance framework llm.npu [67], whose non-attention parts can run on NPU. To the best of our knowledge, this is the only fully open-source framework running on NPUs. Specifically, we replace the attention operator with shadowNPU and its baselines for prefilling stage, and directly employ full attention on CPU/GPU for decoding since this stage is mainly memory-bound.

5 Evaluation

Devices. We mainly perform the experiments on MI14 [64] and Redmi K60 Champion Edition [49]. MI14 is equipped with Qualcomm 8Gen3 SoC and 16+6GB DRAM; Redmi K60 Champion Edition has 8Gen2 and 16+3GB DRAM. The available NPU and CPU/GPU resources are Hexagon-V75/V73 and a mid-core Cortex-A720/A715 processor, respectively.

Models. We mainly test shadowNPU on mobile LLMs. (1) Qwen2-0.5B/1.5B [47, 48]; (2) PhoneLM-0.5B/1.5B [44, 45]. The details are listed in Table 5. The model weights are quantized by third-party frameworks in the following end-to-end experiments.

Datasets. We test shadowNPU on the following datasets.

(1) *ArxivSum* [18] is a general natural language comprehension task. We report the Rouge-L F1 score.

(2) *DroidCall* [65] is an LLM agent task on mobile system API function calling. We report the top 5 single step complete accuracy.

(3) *Octopus* [17] is an LLM agent task on mobile system API function calling. We report the top 5 function selection accuracy.

Each test data is augmented by in-context learning with 5 samples.

Baselines. We compare shadowNPU to the following design alternatives, with fully available NPU and highly limited CPU/GPU resource, i.e., one middle core of CPU.

(1) *Full Attention on CPU/GPU (C/G-Full)* runs float32 full attention on CPU/GPU, a default setting of mainstream on-device inference frameworks, such as llm.npu [67], mlc-llm [39] and HeteroLLM [16].

(2) *Sparse Attention on CPU/GPU (C/G-Sparse)* runs float32 dynamic sparse attention on CPU/GPU, including the dense estimation procedure. This is a vanilla setting of fine-grained dynamic sparse attention [59, 85].

(3) *Block Sparse Attention on CPU/GPU (C/G-Block-Sparse)* runs float32 dynamic sparse attention with a 64x64 block size, which is a common setting in previous literature [73].

(4) *Full Attention on NPU (NPU-Full)* runs INT8 full attention on NPU, with static graph and per-tensor quantization [71].

Settings. In the following experiments, we set global sparsity ratio as 20%, and the number of scale buckets as 9 (step size 5e-1). The rationale of selecting them is detailed in §5.2. The default available CPU/GPU resource for LLM inference is set to one mid core of CPU.

5.1 End-to-End Performance

shadowNPU is accurate. We first report shadowNPU’s accuracy on the aforementioned datasets and models against its baselines. As shown in Table 6, shadowNPU is the most accurate alternative compared to the lossless baseline C/G-Full, with only 0.4 pp. This is due to shadowNPU’s well-designed dynamic sparsity, specifically the fine-grained token-level estimation and head-specific sparsity. Other baselines show significant accuracy loss compared to C/G-Full, with 7.4/11.4/18 pp. The reasons are many fold. The basic reason is that sparsity introduces more noise into the model, especially for compact mobile LLMs and datasets with high information density. For C/G-Block-Sparse, the blocked sparsity makes the model more easier to ignore important tokens. For NPU-Full, the limitations of static graphs and per-tensor quantization result in inaccuracy, particularly for the attention operator, where inputs are activations with high fluctuation.

shadowNPU is fast. As shown in Figure 10, we first report the latency of the attention kernel on MI14. We test the kernel with various input length, i.e., 1024/2048/4096. The values of the input tensors are randomized, and the sparsity ratio of each head is set to global ratio. We have the following observations. Compared to the lossless baseline C/G-Full, shadowNPU is up to 6.9× and on average 3.5× faster. Compared to the lossy baselines, shadowNPU is up to 4.0× faster. Note that according to Table 6, employing these baselines for inference will result in significant accuracy loss.

We further report the end-to-end latency with integrating shadowNPU and its baselines into third-party frameworks. By doing so, we can realize NPU-centric inference. The details are mentioned in §4. We

Model	Dataset	C/G-Full	C/G-Sparse	C/G-Block-Sparse	NPU-Full	Ours
PhoneLM -0.5B	ArxivSum	14.7	14.9	10.0	0.0	15.2
	DroidCall	27.5	24.0	25.5	20.5	25.5
	Octopus	64.6	71.3	62.9	24.1	64.0
PhoneLM -1.5B	ArxivSum	11.9	12.1	6.0	0.0	12.0
	DroidCall	20.5	23.5	28.5	19.0	25.5
	Octopus	79.2	75.8	45.5	24.7	75.8
Qwen2 -0.5B	ArxivSum	10.7	11.4	10.8	9.4	11.3
	DroidCall	34.5	34.5	26.0	27.5	37.5
	Octopus	60.6	42.1	37.6	34.8	61.2
Qwen2 -1.5B	ArxivSum	8.5	2.0	4.2	9.1	8.4
	DroidCall	48.0	19.0	15.5	22.5	44.5
	Octopus	61.2	21.9	32.0	34.2	56.1
Average		36.8	29.4	25.4	18.8	36.4

Table 6: The accuracy on various datasets and models.

perform the experiments on three datasets, whose input sequence dominates the inference (3840/4096/1792 for prefill and 50/10/10 for decode, respectively). This is the most common workload pattern on mobile devices. The results are shown in Figure 11. Compared to the lossless baseline C/G-Full, shadowNPU is up to 4.5× and on average 2.9× faster. Compared to the lossy baselines, shadowNPU is up to 4.0× faster.

We briefly discuss the rationales behind the experimental results. For C/G-Sparse, although it enjoys a 20% sparsity, the speedup is relatively low compared to C/G-Full. This is due to the overhead of estimating the to-be-sparsified positions and the top-k operation. For C/G-Block-Sparse, the speedup is higher. This is due to the estimation overhead is lowered by pre-pooling the Q and K input tensors. For NPU-Full, although NPU has high throughput on matrix multiplication, the speedup is somewhat reduced by operations such as softmax and masking. In comparison, shadowNPU achieves the highest speedup. This is because that it minimizes the estimation overhead by offloading the estimation to NPU and overlapping NPU and CPU/GPU execution.

Besides, shadowNPU achieves on-par or even lower performance compared to SoTA NPU framework llm.npu’s native attention. llm.npu supports two types of attentions, i.e., full attention on 4 CPU cores (1 prime core and 3 middle cores) and full attention on GPU. shadowNPU achieves up to 3.0× lower latency with *only one middle CPU core*.

shadowNPU can work well on various devices. We further show the effectiveness of shadowNPU on other devices. In Table 7, we show the inference latency of PhoneLM-0.5B on Redmi K60 Champion Edition. shadowNPU still shows clear speedup against the baseline (2×/1.25×/1.22×, respectively). The speedup is lower than on MI14. This is mainly due to that the Redmi K60 Champion Edition is equipped with V73 series NPU, which is inferior to the V75 NPU on MI14.

shadowNPU is energy efficient. We show the energy consumption of running a single attention kernel with an input length of 1024 on Redmi K60 Champion Edition. The results are obtained from system APIs²³. As shown in Table 8, shadowNPU exhibits up to 7.66×

²/sys/class/power_supply/battery/voltage_now

³/sys/class/power_supply/battery/current_now

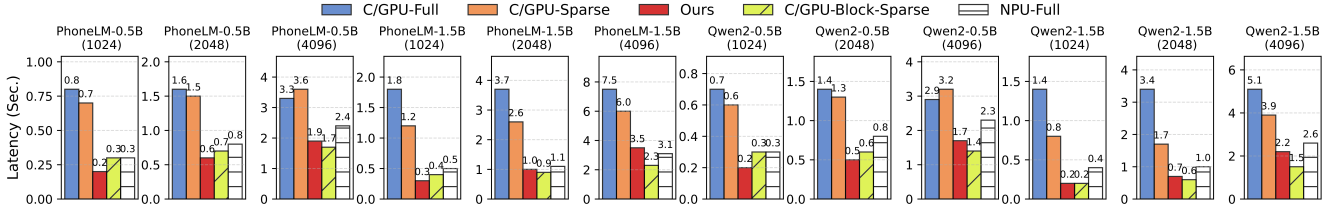


Figure 10: With the same circumstance of highly limited CPU/GPU resources, shadowNPU can achieve much lower attention kernel latency compared to other baselines on MI14.

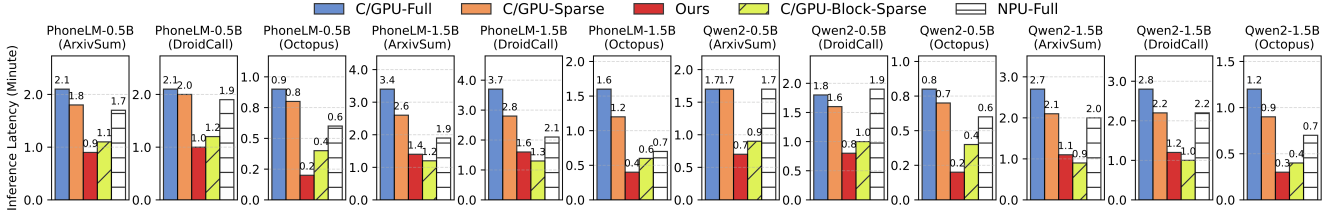


Figure 11: With the same circumstance of highly limited CPU/GPU resources, shadowNPU can achieve much lower end-to-end average inference latency on datasets of real-world mobile tasks compared to other baselines on MI14.

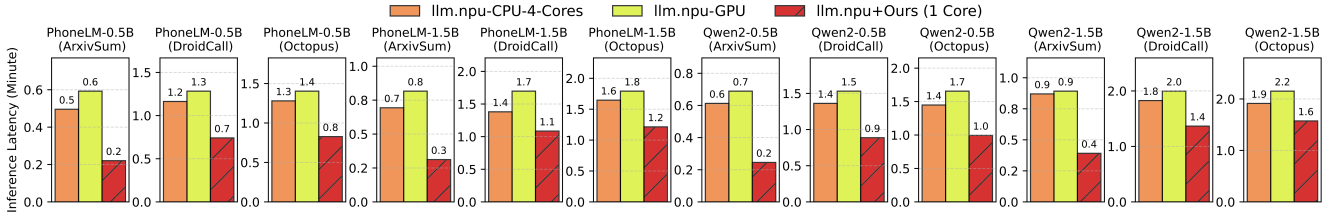


Figure 12: Compared to the native attention in SoTA NPU inference framework that shows high reliance on CPU/GPU, shadowNPU achieves on-par or lower latency with significantly fewer CPU/GPU resources. Device: MI14.

ArxivSum		Octopus		DroidCall	
C/G-Full	Ours	C/G-Full	Ours	C/G-Full	Ours
1.2 Min.	0.6 Min.	2.5 Min.	2.0 Min.	2.7 Min.	2.2 Min.

Table 7: Running shadowNPU on Redmi K60 Champion Edition (Snapdragon 8Gen2). Model: PhoneLM-0.5B.

Model (Single Kernel)	C/G-Full	C/G-Sparse	C/G-Block-Sparse	NPU-Full	Ours
PhoneLM-0.5B	3.72 J	2.81 J	1.10 J	0.85 J	0.66 J
PhoneLM-1.5B	8.59 J	5.16 J	1.51 J	1.29 J	1.12 J
Qwen2-0.5B	3.38 J	2.35 J	0.88 J	0.85 J	0.56 J
Qwen2-1.5B	5.29 J	3.28 J	0.85 J	1.10 J	0.71 J

Table 8: Energy consumption of an attention kernel. Device: Redmi K60 Champion Edition; Input length: 1024.

lower energy consumption. This is primarily due to the reduced computational load and the proper utilization of NPU, which has relatively low power.

Running FP16 on NPUs. A portion of smartphone NPUs do support FP16 precision. However, as we mentioned in Table 1, on one hand, the FP16 is not a common feature across all platforms; on the other hand, on NPUs that support FP16, the performance can also be very low due to the design of hardware and software stack. We show this limitation by a comparison in Table 9. We generate

the FP16 compute-graph by QNN [5]⁴. We test the attention kernel latency on MI14 smartphone (snapdragon 8Gen3 SoC), with PhoneLM-0.5B/1.5B models. The context length is 2048. For FP16 attention on NPUs, the latency is 82 seconds on PhoneLM-0.5B and 133 seconds on PhoneLM-1.5B — over 100× slower than our system. The results show that the FP16 ability of NPUs is not the main focus

⁴Although there is assembly-level hacking [22] that better unleashes the FP16 compute of Qualcomm NPUs, such hacking is not officially supported or encouraged. The newly released NPU versions can also be incompatible to it.

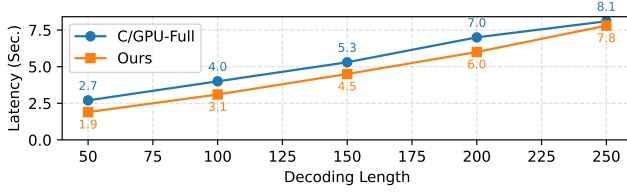


Figure 13: Performance under decoding dominant workload. Prefill length: 256; Model: PhoneLM-1.5B; Device: MI14.

under the current official environment. Such an observation is also backed up by recent work [67].

Moderate prefill length. The scenario this work focuses is representative on mobile devices — applications with long prefill length. For example, the API calling agents, which are killer apps, require processing the document of candidate functions. The prefill length is typically thousands of tokens, dominating the end-to-end inference latency. With moderate prefill length that no longer dominates the end-to-end inference, our system still works. In Figure 13, we show the performance of our system when the decode length rises (50–200 tokens). On MI14 and PhoneLM-1.5B, when we prefill 256 tokens and decode 50 tokens, the end-to-end latency of C/GPU-Full is 2.7 seconds, and shadowNPU is 1.9 seconds. The speedup is still over 1.4×. Since the sparsity is currently only applied to prefill stage (§4), the speedup diminishes with the decoding length. We leave the acceleration of the decoding stage as the future work.

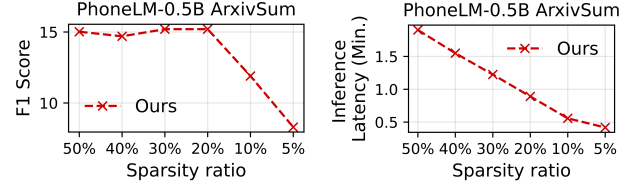
More realistic accuracy. Considering that some applications in production environments require a low error rate, we provide the following add-on tests. We add three new datasets (ChnSentiCorp, SST-2 and LlamaTouch) and one new model (Qwen2-7B). ChnSentiCorp [10] is a benchmark dataset for Chinese sentiment analysis. It comprises hotel, book, and laptop reviews, labeled positive or negative, widely used for NLP applications. SST-2 [52] is a benchmark for English sentiment analysis. It features movie reviews labeled positive or negative. LlamaTouch [82] is mobile app using dataset. It consists of mobile app calling traces for LLM agents. As shown in Table 10, the accuracy drop is only 1–3 pp (with over 70/90 pp absolute accuracy), with Qwen2-7B on ChnSentiCorp and SST-2. The accuracy on LlamaTouch is over 40 pp, which we believe can be higher with the evolution of model.

	NPU-Full-FP16	CPU/GPU-Full	Ours
0.5B	82	1.6	0.6
1.5B	133	3.7	1.0

Table 9: Comparing to NPU FP16 by attention kernel latency (seconds). Device: MI14 (snapdragon 8Gen3). Context length: 2048. Model: PhoneLM.

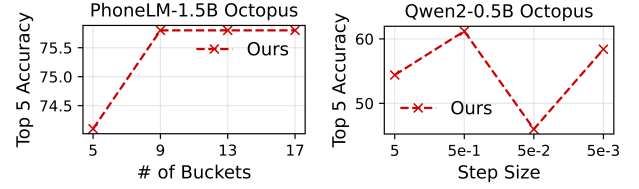
	Full	SWA	TM	Ours
CSC	0.77	0.47	0.63	0.76
SST-2	0.94	0.53	0.93	0.91
LT	0.44	0.19	0.2	0.43

Table 10: Add-on experiments. We test Qwen2-7B on ChnSentiCorp [10], SST-2 [52] and LlamaTouch [82]. The sliding window attention (SWA) and TraingleMix [23] (TM) use a sparsity ratio of 20%.



(a) Sparsity ratio v.s. accuracy. (b) Sparsity ratio v.s. latency.

Figure 14: Sensitivity analysis of global sparsity ratio.



(a) Sensitivity to # of buckets. (b) Sensitivity to step size.

Figure 15: Sensitivity analysis of scale factor buckets.

Comparing to static sparsity. We also compare our system to static sparsity. In Table 10, we employ static sparse patterns of sliding window attention (SWA) and TraingleMix [23] (TM). The window size and triangle size (in average with dense layers) are set to the sparsity ratio (20%). The accuracy drop is significant, e.g., 30 pp on ChnSentiCorp, 41 pp on SST-2 and 24 pp on LlamaTouch for sliding window attention. The rationale behind it is that the static sparsity cannot capture fine-grained patterns. In comparison, the dynamic sparsity of our system is more flexible. It can capture token/block level sparsity, without any pre-defined patterns. For TraingleMix, to the best of our knowledge, we did not find the open source version. As a result, we reproduced it based on the illustration of the paper. Although better than sliding window attention, TraingleMix still undergoes an average accuracy drop of 13 pp on the three datasets. The rationale behind it is mainly that TraingleMix only applies a single pattern of static sparsity, i.e., triangle sparsity, which is mixed with simple full attention across the layers, being inflexible. Besides, the layer-level profiling of TraingleMix is coarse-grained for capturing the sparsity compared to our head-level solution.

5.2 Sensitivity Analysis and Ablation Study

Global sparsity ratio. We show the sensitivity of sparsity ratio in Figure 14. Setting the ratio to 20% is a reasonable choice. A too large ratio (e.g. 30%, 40%, or 50%) reduces less latency; a too small ratio leads to significant accuracy loss. 20% is the minimum sparsity ratio that maintains no accuracy loss, i.e., the knee point. As a result, shadowNPU chooses 20% as its default setting, which also aligns well with the aforementioned analysis in §3.

Scale factor buckets. Recall that we cache subgraphs of various scales on NPU for Q·K. In shadowNPU’s default setting, the number of buckets is set to 9 with a step size of 5e-1. More buckets offer a more fine-grained tradeoff space, which leads to higher accuracy

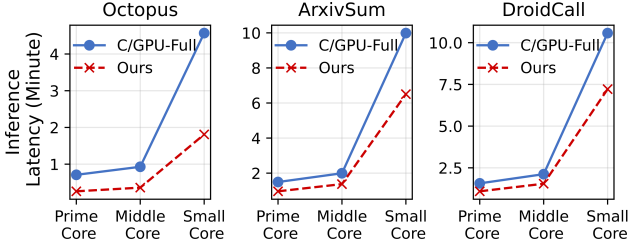


Figure 16: Varying the available resource of CPU/GPU.

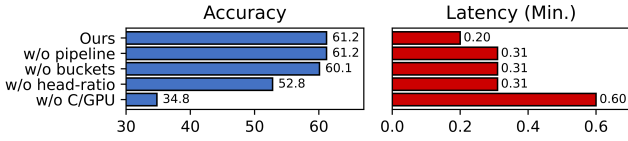


Figure 17: Ablation study on Qwen2-0.5B, MI14, Octopus.

(5 buckets to 9 buckets in Figure 15a); yet the number of buckets exhibits a marginal effect that adding more buckets shows almost no performance gain. As a result, typically we choose it as 9. Figure 15b shows why we choose a step size of $5e-1$. Factors of other orders of magnitude lead to significant performance degradation. Interestingly, a very tiny step size ($5e-3$) results in less accuracy drop (yet still significant). This is primarily because that such a choice makes all samples fall back into the original bucket, since the scale factors of others are too large or too small. In a nut shell, the default setting of shadowNPU on the buckets is reasonable and leads to the best performance.

The available CPU/GPU. Recall that in our default setting, we set the available CPU/GPU resource to a middle core of ARM CPU. We now vary the available resource in Figure 16 by mapping it to cores with various compute capability. The experiment is run on MI14, which has 1 prime core, 5 middle cores, and 2 small cores. Firstly, shadowNPU consistently outperforms C/G-Full. Besides, when expanding the compute resource (e.g., small core to middle core to prime core), the absolute inference latency of shadowNPU is also reduced, showing strong scalability. Overall, shadowNPU is a robust solution under various amount of compute resource.

Ablation study. We conduct ablation study by gradually removing the key designs of shadowNPU. The results are shown in Figure 17. When the pipeline is removed from shadowNPU, the end-to-end latency exhibits a significant rise. This is because that the pipeline overlaps the latency of NPU and CPU/GPU. On top of this, when we further remove the design of scale factor buckets, the accuracy shows a clear drop (1.2 pp). This is because the buckets allow the NPU to adapt better to the input range. Further, when we make each head share the same global sparsity ratio, the accuracy drops from 60.1 pp to 52.8 pp. The rationale is that the intrinsic importance of each head is ignored. Finally, when we even cancel the design of sparse compute on CPU/GPU and directly offloading the full attention to NPU, the accuracy is lowered to 34.8 pp, and the latency is moved to 0.6 minutes. This is because shadowNPU have to suffer the quantization loss on NPU, and the NPU is not friendly to sparse

App	①	②	③	④	⑤
Latency	14.7 Sec.	14.5 Sec.	15.5 Sec.	14.7 Sec.	15.2 Sec.

Table 11: Running shadowNPU with other apps. Model: PhoneLM-0.5B; Dataset: Octopus; Device: MI14. ①: No concurrent apps; ②: Taking photos; ③: Watching videos; ④: Listening to music; ⑤: Browsing shopping apps.

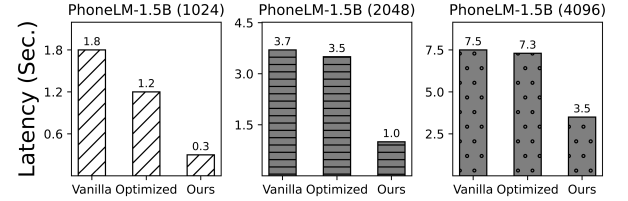


Figure 18: Ablating the implementation optimizations. Device: MI14. Vanilla: C/GPU Full attention with framework-native implementation; Optimized: C/GPU Full attention with our proposed implementation optimizations.

compute. In conclusion, each part of shadowNPU shows non-trivial contribution to its performance.

Running alongside other mobile apps. We show shadowNPU’s superiority by running the LLM inference together with other mobile apps. Specifically, we test the end-to-end inference time when the smartphone user is watching videos (GPU and CPU busy), listening to music (CPU busy), tasking photos (ISP and CPU busy) and browsing shopping apps (CPU busy). We exclude other apps for running on the selected middle core. The results are shown in Table 11. The LLM inference can work well with these apps, with no obvious performance degradation.

Ablating the implementation optimizations. The vanilla baseline (i.e., C/GPU-Full) directly reuses the native attention implementation of the on-device LLM inference framework. In our implementation, we do several optimizations (§4). We use half-precision vector reduction in neon, since accumulating the few sparse elements is less likely to overflow. Besides, for the easier scheduling of NPU-CPU/GPU pipeline, we fuse the operators to create a whole C/GPU sparse QKV compute graph, reducing the read/write of DRAM. Our key techniques (e.g., pipeline and sparsification) contribute most of the performance improvements. The results are shown in Figure 18. As shown in the figure caption, the “Vanilla” means the framework-native implementation of C/GPU full attention; the “Optimized” means transferring the implementation optimizations of our system to C/GPU full attention. The performance improvement introduced by the implementation is limited (i.e., 2.6%–33%). For shorter context length, the improvement is relatively higher, due to the higher proportion of compute graph launching overhead, etc. The performance gain from our design is still $2.08\times-4\times$.

6 Discussion

Generalization beyond a single NPU family. As far as we know, the Qualcomm NPU is widely used, high-performance, and its

software stack is open-to-modification. As a result, we choose it as our testbed. However, our system is not only limited to Qualcomm NPUs. With the same design philosophy, shadowNPU can also be transferred to NPUs like Huawei or MediaTek NPUs. For instance, Huawei Ascend 310 features a similar software stack (e.g., static shape; compiler based [78]) and hardware stack (CPU/GPU/NPU family) compared to Qualcomm. Thereby, the efforts needed for transferring are also minimized. The developers only need to generate the compute graphs of Ascend NPU, do the bucketing, profiling and pipeline planning, with almost the same workflow.

Comparison to sparse attention for other models. There are also attentions for variants of Transformers. For instance, Sparse VideoGen [61] also employs sparse attention for Transformer-based models. However, it targets on Diffusion Transformers (DiTs) that generates videos. Videos have two levels of characteristic — spatial and temporal. Thus, Sparse VideoGen designs specific static sparse patterns for spatial head and temporal head. However, LLMs do not have these features; its input and output are plain texts. As a result, Sparse VideoGen is not suitable for the on-device LLM inference acceleration scenario.

7 Related Work

On-Device LLM Inference. Running mobile LLMs [11, 28, 30, 36, 54, 58, 83, 84] on devices has become a key area in privacy-preserving mobile intelligence [14, 29, 40, 43, 51]. There exist many popular frameworks. For instance, HeteroLLM [16] proposes a GPU/NPU collaborated framework for LLM inference on mobile SoCs. It optimizes the memory copy efficiency. However, the attention and parts of MLP still run on GPU, which can be busy during the usage of mobile devices. Akin to HeteroLLM, llm.npu [67] runs attention on CPU/GPU, which cannot realize the NPU-centric inference. PowerinferV2 [71] employs dense attention on NPUs, i.e., the NPU-Full baseline. However, compared to shadowNPU, it does not support sparse attention. Even worse, it still suffers from the accuracy drop of attention quantization, especially on mobile-sized LLMs and mobile agent tasks. Its main focus is on extending the RAM of mobile devices to run large-scale LLMs that have stronger resilience to attention quantization. Akin to PowerinferV2, QNN [5] also runs full attention on NPU, encountering the same problems. There are also frameworks that run both the attention and non-attention parts on CPU/GPU. For example, MLC-LLM [39] and TFLite [57] run LLMs on GPUs; Llama.cpp [21] runs LLMs on CPUs. By only keeping the sparse compute of important tokens on CPU/GPU, shadowNPU realizes the most accurate and efficient NPU-centric inference.

Efficient Sparse Attention. Fixed-pattern sparse attention [25–27, 63] identifies one of given patterns offline. Common patterns include sliding window, Λ -shape and vertical-slash, etc. Compared to ad-hoc fixed patterns, shadowNPU’s dynamic attention is a superset of the above patterns. Block sparse attention [56, 70, 73, 80] pre-pools the Q and K, and selects top k areas in coarse-grained block level. As discussed before, such a paradigm is less accurate. Reserving/discarding a whole region may dismiss important tokens. shadowNPU’s fine-grained sparse attention is much more accurate. Recently there is also a trend of training native sparse attentions [35, 79]. Such methods make the attention inherently sparse

at the inference stage. However, the learned sparsity cannot directly transfer to downstream tasks and fine-tuned models. The resource consumption of pre-training is also not affordable to each developer. In comparison, the sparse attention of shadowNPU is plug-and-play. **Mobile Intelligence.** Mobile intelligence runs DNNs especially foundation models on edge/mobile devices for sensing the human/environment or acting with the cyber/physical world [13, 32, 41, 66, 68, 69, 72, 75–77]. With shadowNPU’s NPU-centric inference, the execution of foundation models will be more transparent and less intrusive to the human and environment.

8 Conclusion

We present shadowNPU, a sparse attention module for NPU-centric on-device LLM inference. It incorporates innovative techniques including NPU-based estimation, NPU compute graph bucketing and NPU-CPU/GPU pipeline. It losslessly achieves up to 4.5× end-to-end speed up while minimizing the reliance on CPU/GPU resources.

Acknowledgments

This work was supported by National Natural Science Foundation of China under the grant number 62325201. Mengwei Xu is supported by NSFC (No. 62522202) and Beijing Natural Science Foundation (No. L253005).

A Artifact Appendix

A.1 Abstract

Running LLMs on devices like smartphones has become a key catalyst towards privacy-preserving mobile AI. In state-of-the-art frameworks, the attention operator falls back from the dedicated NPU to the public CPU/GPU due to its sensitivity to quantization. Such a fallback leads to hampered user experience and extra system scheduling complexity. To realize NPU-centric LLM inference, this paper presents shadowNPU, a system-algorithm codesigned sparse attention module with minimal reliance on CPU/GPU by only sparsely computing a very small number of tokens. The key idea is to hide the overhead of estimating the important tokens by offloading it to NPU. On top of this, it further incorporates insightful techniques including NPU compute graph bucketing, head-wise NPU-CPU/GPU pipeline and per-head fine-grained sparsity ratio to achieve high accuracy and efficiency. Compared to design alternatives, shadowNPU achieves the best performance with strictly limited CPU/GPU resource; it requires much less CPU/GPU resource to achieve on-par performance of SoTA frameworks.

A.2 Artifact check-list (meta-information)

- **Model:** PhoneLM-0.5B, PhoneLM-1.5B, Qwen2-0.5B, Qwen2-1.5B
- **Dataset:** ArxivSum, DroidCall, Octopus
- **Run-time environment:** Ubuntu22, Android
- **Hardware:** Cloud A100 GPU server, MI14 smartphone, Redmi K60 Champion Edition
- **Execution:** Python and bash scripts; C++ binaries
- **Metrics:** Accuracy, latency
- **Output:** Logs and figures
- **Experiments:** (1) Testing the performance of our method against the baselines on cloud. (2) Testing the latency on devices.

- How much disk space required (approximately)?: over 500GB
- How much time is needed to prepare workflow (approximately)?: 1 hour
- How much time is needed to complete experiments (approximately)?: over 24 hours
- Publicly available?: Yes
- Code licenses (if publicly available)?: MIT
- Data licenses (if publicly available)?: MIT
- Archived (provide DOI)?: Not yet

A.3 Description

A.3.1 How to access. Considering that the environment of our system is somehow hard to prepare, we pack it into the docker image, which is available in Aliyun⁵.

The code is archived in Zenodo⁶. The open source version will be released in github⁷.

A.3.2 Preparation. We recommend you directly using the docker image, or checking the software version details in the docker image and manually installing them.

```
docker pull crpi-l2n42pu76ihl4t2x.cn-beijing.personal.cr.aliyuncs.com/yinwangsong2000/shadownpu-ae
```

Then, enter the docker container and go ahead for the following.

A.3.3 Experiments on cloud servers. Typically, you should install the modified third-party libs such as 3rdparty and transformer_patch by

```
pip install -e .
```

However, in our docker image, such things have been completed.

Your HF_HOME has also been set as “/data/share”. You may need a double-check.

The head sparsity ratio and layer importance are profiled in the directory of “shadowNPU/offline”. The profiling scripts are located in “/root/ShadowNPU/shadowNPU/offline/scripts/head_imp_offline_profiling.sh” and so on. You may need to check the code that is turned into comment in advance. The profiled results will be stored in the directory of “/root/ShadowNPU/shadowNPU/offline/results/” and “/root/ShadowNPU/shadowNPU/offline/ratios/”.

The results have been pre-profiled. You do not need to do anything more in this docker image.

To reproduce the results of accuracy, e.g., table 6, you need to run the scripts in “/root/ShadowNPU/shadowNPU/scripts/”. You can run all three datasets by

```
bash /root/ShadowNPU/shadowNPU/scripts/run_all.sh 0
```

You can also run per-dataset experiment by

```
bash /root/ShadowNPU/shadowNPU/scripts/run_arxivsum.sh 0
bash /root/ShadowNPU/shadowNPU/scripts/run_droidcall.sh 0
bash /root/ShadowNPU/shadowNPU/scripts/run_octopus.sh 0
```

The results will be stored in “/root/ShadowNPU/shadowNPU/results”.

⁵<https://cr.console.aliyun.com/repository/cn-beijing/yinwangsong2000/shadownpu-ae>

⁶<https://doi.org/10.5281/zenodo.19555734>

⁷<https://github.com/UbiquitousLearning/mllm>

A.3.4 Experiments on smartphones. Firstly, you need to generate the compute graph of NPU in “/root/ShadowNPU/shadowNPU/mllm/src/backends/qnn/sdk/examples/QNN/SampleApp”. Please check the readme.md and generate the .bin files.

We have pre-generated the files.

Then, build the inference framework. Run

```
/root/ShadowNPU/shadowNPU/mllm/scripts/build_android_qnn.sh
```

Get the end-to-end results by

```
/root/ShadowNPU/shadowNPU/scripts/run_e2e_experiments.sh
```

The results will be located in “/root/ShadowNPU/shadowNPU/results/e2e.txt”. The data leads to the Figure 11.

Get the kernel results by running the scripts in

```
/root/ShadowNPU/shadowNPU/single_kernels/scripts
```

The results are in

```
/root/ShadowNPU/shadowNPU/single_kernels/res
```

For the sensitivity analysis of figure 13–16, you need to modify the code at the corresponding positions. We have not prepared an end-to-end script.

A.3.5 Drawing the figures. We have run all the experiments beforehand. If you want to reproduce the results quickly, please directly refer to “/root/ShadowNPU/shadowNPU/drawing/drawing.ipynb”.

References

- [1] 2025. ARM NEON. <https://www.arm.com/technologies/neon>.
- [2] 2025. Hexagon NPU SDK. <https://www.qualcomm.com/developer/software/hexagon-npu-sdk>.
- [3] 2025. LLVM. <https://llvm.org/>.
- [4] 2025. Open CL. <https://en.wikipedia.org/wiki/OpenCL>.
- [5] 2025. QNN SDK. <https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-50/introduction.html>.
- [6] 2025. Qualcomm Neural Processing Engine. <https://docs.qualcomm.com/bundle/publicresource/topics/80-70015-15BY/snpe.html>.
- [7] 2025. rewind. <https://www.rewind.ai/>.
- [8] 2025. Snapdragon 8 gen 3 mobile platform product brief. https://docs.qualcomm.com/bundle/publicresource/87-71408-1_REV_C_Snapdragon_8_gen_3_Mobile_Platform_Product_Brief.pdf.
- [9] 2025. TMS320F2812 platform product brief. <https://www.ti.com/product/TMS320F2812>.
- [10] 2026. ChnSentiCorp dataset. <https://huggingface.co/datasets/lansinuoet/ChnSentiCorp>.
- [11] Marah Abidin and etc. Jyoti Aneja. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219 [cs.CL] <https://arxiv.org/abs/2404.14219>
- [12] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL] <https://arxiv.org/abs/2309.16609>
- [13] Ozan Baris, Yizhuo Chen, Gaofeng Dong, Liying Han, Tomoyoshi Kimura, Pengrui Quan, Ruijie Wang, Tianchen Wang, Tarek Abdelzaher, Mario Bergés, Paul Pu Liang, and Mani Srivastava. 2025. Foundation Models for CPS-IoT: Opportunities and Challenges. arXiv:2501.16368 [cs.LG] <https://arxiv.org/abs/2501.16368>
- [14] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. arXiv:2506.02153 [cs.AI] <https://arxiv.org/abs/2506.02153>
- [15] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. 2025. A Survey on Mixture of Experts in Large Language Models. *IEEE Transactions on Knowledge and Data Engineering* (2025), 1–20. doi:10.1109/tkde.2025.3554028
- [16] Le Chen, Dahu Feng, Erhu Feng, Rong Zhao, Yingrui Wang, Yubin Xia, Haibo Chen, and Pinjie Xu. 2025. HeteroLLM: Accelerating Large Language

- Model Inference on Mobile SoCs platform with Heterogeneous AI Accelerators. arXiv:2501.14794 [cs.DC] <https://arxiv.org/abs/2501.14794>
- [17] Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. arXiv:2404.01744 [cs.CL] <https://arxiv.org/abs/2404.01744>
 - [18] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. 2018. A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Association for Computational Linguistics, New Orleans, Louisiana, 615–621. doi:10.18653/v1/N18-2097
 - [19] Ziyang Fu, Ju Ren, Deyu Zhang, Yuezhong Zhou, and Yaoxue Zhang. 2022. Kalmia: A Heterogeneous QoS-aware Scheduling Framework for DNN Tasks on Edge Servers. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. 780–789. doi:10.1109/INFOCOM48880.2022.9796661
 - [20] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. arXiv:2310.01801 [cs.CL] <https://arxiv.org/abs/2310.01801>
 - [21] ggml. 2025. *llama.cpp*. <https://github.com/ggml-org/llama.cpp>
 - [22] Zixu Hao, Jianyu Wei, Tuowei Wang, Minxing Huang, Huiqiang Jiang, Shiqi Jiang, Ting Cao, and Ju Ren. 2025. Scaling LLM Test-Time Compute with Mobile NPU on Smartphones. arXiv preprint arXiv:2509.23324 (2025).
 - [23] Zhiyuan He, Yike Zhang, Chengruidong Zhang, Huiqiang Jiang, Yuqing Yang, and Lili Qiu. 2025. TriangleMix: Accelerating Prefilling via Decoding-time Contribution Sparsity. arXiv:2507.21526 [cs.CL] <https://arxiv.org/abs/2507.21526>
 - [24] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. 2022. Band: coordinated multi-DNN inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (Portland, Oregon) (MobiSys '22)*. Association for Computing Machinery, New York, NY, USA, 235–247. doi:10.1145/3498361.3538948
 - [25] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
 - [26] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. arXiv:2407.02490 [cs.CL] <https://arxiv.org/abs/2407.02490>
 - [27] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. 2025. FlexPre-fill: A Context-Aware Sparse Attention Mechanism for Efficient Long-Sequence Inference. arXiv:2502.20766 [cs.LG] <https://arxiv.org/abs/2502.20766>
 - [28] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steven Y. Ko, Sangeun Oh, and Insik Shin. 2024. Explore, Select, Derive, and Recall: Augmenting LLM with Human-like Memory for Mobile Task Automation. arXiv:2312.03003 [cs.HC] <https://arxiv.org/abs/2312.03003>
 - [29] Liang Li, Xingke Yang, Wen Wu, Hao Wang, Tomoaki Ohtsuki, Xin Fu, Miao Pan, and Xuemin Shen. 2025. MobilLLM: Enabling LLM Fine-Tuning on the Mobile Device via Server Assisted Side Tuning. arXiv:2502.20421 [cs.LG] <https://arxiv.org/abs/2502.20421>
 - [30] Xiang Li, Zhenyan Lu, Dongqi Cai, Xiao Ma, and Mengwei Xu. 2024. Large Language Models on Mobile Devices: Measurements, Analysis, and Insights. In *Proceedings of the Workshop on Edge and Mobile Foundation Models (Minato-ku, Tokyo, Japan) (EdgeFM '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3662006.3662059
 - [31] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978 [cs.CL] <https://arxiv.org/abs/2306.00978>
 - [32] Kaiwei Liu, Bufang Yang, Lilin Xu, Yunqi Guo, Guoliang Xing, Xian Shuai, Xiaozhe Ren, Xin Jiang, and Zhenyu Yan. 2025. TaskSense: A Translation-like Approach for Tasking Heterogeneous Sensor Systems with LLMs. Association for Computing Machinery, New York, NY, USA, 213–225. <https://doi.org/10.1145/3715014.3722070>
 - [33] Mukul Lokhande, Gopal Raut, and Santosh Kumar Vishvakarma. 2024. FlexPE: Flexible and SIMD Multi-Precision Processing Element for AI Workloads. arXiv:2412.11702 [cs.AR] <https://arxiv.org/abs/2412.11702>
 - [34] Mukul Lokhande and Santosh Kumar Vishvakarma. 2025. POLARON: Precision-aware On-device Learning and Adaptive Runtime-cOnfigurable AI acceleration. arXiv:2506.08785 [cs.AR] <https://arxiv.org/abs/2506.08785>
 - [35] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Fan, Jianlin Su, Yuxin Wu, Neo Y. Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. 2025. MoBA: Mixture of Block Attention for Long-Context LLMs. arXiv:2502.13189 [cs.LG] <https://arxiv.org/abs/2502.13189>
 - [36] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764 [cs.CL] <https://arxiv.org/abs/2402.17764>
 - [37] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843 [cs.CL]
 - [38] Melkamu Mersha, Khang Lam, Joseph Wood, Ali K. AlShami, and Jugal Kalita. 2024. Explainable artificial intelligence: A survey of needs, techniques, applications, and future direction. *Neurocomputing* 599 (Sept. 2024), 128111. doi:10.1016/j.neucom.2024.128111
 - [39] MLC team. 2023–2025. *MLC-LLM*. <https://github.com/mlc-ai/mlc-llm>
 - [40] Mozhgan Navardi, Romina Aalishah, Yuzhe Fu, Yueqian Lin, Hai Li, Yiran Chen, and Timosh Mohsenin. 2025. GenAI at the Edge: Comprehensive Survey on Empowering Edge Devices. arXiv:2502.15816 [cs.DC] <https://arxiv.org/abs/2502.15816>
 - [41] Xiaomin Ouyang, Xian Shuai, Yang Li, Li Pan, Xifan Zhang, Heming Fu, Sitong Cheng, Xinyan Wang, Shihua Cao, Jiang Xin, Hazel Mok, Zhenyu Yan, Doris Sau Fung Yu, Timothy Kwok, and Guoliang Xing. 2024. ADMAR: A Multi-Modal Federated Learning System for Monitoring Digital Biomarkers of Alzheimer's Disease. arXiv:2310.15301 [cs.LG] <https://arxiv.org/abs/2310.15301>
 - [42] Jun-Seok Park, Changsoo Park, Suknam Kwon, Taeho Jeon, Yesung Kang, Heonsoo Lee, Dongwoo Lee, James Kim, Hyeon-Seok Kim, YoungJong Lee, Sangkyu Park, MinSeong Kim, SangHyuck Ha, Jihoon Park, Jinpyo Park, SukHwan Lim, and Inyup Kang. 2023. A Multi-Mode 8k-MAC HW-Utilization-Aware Neural Processing Unit With a Unified Multi-Precision Datapath in 4-nm Flagship Mobile SoC. *IEEE Journal of Solid-State Circuits* 58, 1 (2023), 189–202. doi:10.1109/JSSC.2022.3205713
 - [43] Dan Peng, Zhihui Fu, and Jun Wang. 2024. PocketLLM: Enabling On-Device Fine-Tuning for Personalized LLMs. arXiv:2407.01031 [cs.LG] <https://arxiv.org/abs/2407.01031>
 - [44] phonelm. 2025. *PhoneLM-0.5B*. <https://huggingface.co/unisloth/PhoneLM-0.5B>
 - [45] phonelm. 2025. *PhoneLM-1.5B*. <https://huggingface.co/unisloth/PhoneLM-1.5B>
 - [46] Qwen, , An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuxiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
 - [47] qwen. 2025. *Qwen2-0.5B*. <https://huggingface.co/unisloth/Qwen2-0.5B>
 - [48] qwen. 2025. *Qwen2-1.5B*. <https://huggingface.co/unisloth/Qwen2-1.5B>
 - [49] redmi. 2025. *Redmi K60 Champion Edition Smartphone*. https://www.gsmarena.com/xiaomi_redmi_k60_pro-12046.php
 - [50] Tanmoy Sen, Haiying Shen, and Anand Padmanabha Iyer. 2025. Flex: Fast, Accurate DNN Inference on Low-Cost Edges Using Heterogeneous Accelerator Execution. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 507–523. doi:10.1145/3689031.3696067
 - [51] Andrii Skliar, Ties van Rozendaal, Romain Lepert, Todor Bojinovski, Mart van Baalen, Markus Nagel, Paul Whatmough, and Babak Ehteshami Bejnordi. 2025. Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference. arXiv:2412.00099 [cs.LG] <https://arxiv.org/abs/2412.00099>
 - [52] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Seattle, Washington, USA, 1631–1642. <https://www.aclweb.org/anthology/D13-1170>
 - [53] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2023. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864 [cs.CL] <https://arxiv.org/abs/2104.09864>
 - [54] Shreyas Subramanian, Vikram Elango, and Mecit Gungor. 2025. Small Language Models (SLMs) Can Still Pack a Punch: A survey. arXiv:2501.05465 [cs.CL] <https://arxiv.org/abs/2501.05465>
 - [55] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLoS '25)*. Association for Computing Machinery, New York, NY, USA, 1302–1316. doi:10.1145/3676641.3716278
 - [56] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. arXiv:2406.10774 [cs.CL] <https://arxiv.org/abs/2406.10774>
 - [57] TFLite team. 2025. *mediapipe*. <https://ai.google.dev/edge/mediapipe/solutions/guide>
 - [58] Omkar Thawakar, Ashmal Vayani, Salman Khan, Hisham Cholakov, Rao M. Anwer, Michael Felsberg, Tim Baldwin, Eric P. Xing, and Fahad Shahbaz Khan.

2024. *MobiLlama: Towards Accurate and Lightweight Fully Transparent GPT*. arXiv:2402.16840 [cs.CL] <https://arxiv.org/abs/2402.16840>
- [59] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. *HPCA* (2021).
- [60] Jianyu Wei, Ting Cao, Shijie Cao, Shiqi Jiang, Shaowei Fu, Mao Yang, Yanyong Zhang, and Yunxin Liu. 2023. NN-Stretch: Automatic Neural Network Branching for Parallel Inference on Heterogeneous Multi-Processors. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services* (Helsinki, Finland) (*MobiSys '23*). Association for Computing Machinery, New York, NY, USA, 70–83. doi:10.1145/3581791.3596870
- [61] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, Jianfei Chen, Ion Stoica, Kurt Keutzer, and Song Han. 2025. Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity. arXiv:2502.01776 [cs.CV] <https://arxiv.org/abs/2502.01776>
- [62] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2024. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438 [cs.CL] <https://arxiv.org/abs/2211.10438>
- [63] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. arXiv:2410.10819 [cs.CL] <https://arxiv.org/abs/2410.10819>
- [64] xiaomi. 2025. *MI14 Smartphone*. <https://www.mi.com/global/product/xiaomi-14/specs/>
- [65] Weikai Xie, Li Zhang, Shihe Wang, Rongjie Yi, and Mengwei Xu. 2024. DroidCall: A Dataset for LLM-powered Android Intent Invocation. arXiv:2412.00402 [cs.AI] <https://arxiv.org/abs/2412.00402>
- [66] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2025. EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding. *IEEE Transactions on Mobile Computing* 24, 4 (2025), 3256–3273. doi:10.1109/TMC.2024.3513457
- [67] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2025. Fast On-device LLM Inference with NPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (*ASPLOS '25*). Association for Computing Machinery, New York, NY, USA, 445–462. doi:10.1145/3669940.3707239
- [68] Mengwei Xu, Dongqi Cai, Wangsong Yin, Shangguang Wang, Xin Jin, and Xuanzhe Liu. 2025. Resource-efficient Algorithms and Systems of Foundation Models: A Survey. *ACM Comput. Surv.* 57, 5, Article 110 (Jan. 2025), 39 pages. doi:10.1145/3706418
- [69] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, Qiyang Zhang, Zhenyan Lu, Li Zhang, Shangguang Wang, Yuanchun Li, Yunxin Liu, Xin Jin, and Xuanzhe Liu. 2024. A Survey of Resource-efficient LLM and Multimodal Foundation Models. arXiv:2401.08092 [cs.LG] <https://arxiv.org/abs/2401.08092>
- [70] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. 2025. XAttention: Block Sparse Attention with Antidiagonal Scoring. arXiv:2503.16428 [cs.CL] <https://arxiv.org/abs/2503.16428>
- [71] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. arXiv:2406.06282 [cs.LG] <https://arxiv.org/abs/2406.06282>
- [72] Bufang Yang, Lilin Xu, Liekang Zeng, Kaiwei Liu, Siyang Jiang, Wenrui Lu, Hongkai Chen, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. 2025. ContextAgent: Context-Aware Proactive LLM Agents with Open-World Sensory Perceptions. arXiv:2505.14668 [cs.AI] <https://arxiv.org/abs/2505.14668>
- [73] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025. LServe: Efficient Long-sequence LLM Serving with Unified Sparse Attention. arXiv:2502.14866 [cs.CL] <https://arxiv.org/abs/2502.14866>
- [74] Juheon Yi and Youngki Lee. 2020. Heimdall: mobile GPU coordination platform for augmented reality applications. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking* (London, United Kingdom) (*MobiCom '20*). Association for Computing Machinery, New York, NY, USA, Article 35, 14 pages. doi:10.1145/3372224.3419192
- [75] Wangsong Yin, Daliang Xu, Gang Huang, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2024. PieBridge: Fast and Parameter-Efficient On-Device Training via Proxy Networks. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems* (Hangzhou, China) (*SenSys '24*). Association for Computing Machinery, New York, NY, USA, 126–140. doi:10.1145/3666025.3699327
- [76] Wangsong Yin, Mengwei Xu, Yuanchun Li, and Xuanzhe Liu. 2024. LLM as a System Service on Mobile Devices. arXiv:2403.11805 [cs.OS] <https://arxiv.org/abs/2403.11805>
- [77] Wangsong Yin, Rongjie Yi, Daliang Xu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2024. ELMS: Elasticized Large Language Models On Mobile Devices. arXiv:2409.09071 [cs.DC] <https://arxiv.org/abs/2409.09071>
- [78] Feng Yu, Guangli Li, Jiacheng Zhao, Huimin Cui, Xiaobing Feng, and Jingling Xue. 2024. Optimizing Dynamic-Shape Neural Networks on Accelerators via On-the-Fly Micro-Kernel Polymerization. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (*ASPLOS '24*). Association for Computing Machinery, New York, NY, USA, 797–812. doi:10.1145/3620665.3640390
- [79] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv:2502.11089 [cs.CL] <https://arxiv.org/abs/2502.11089>
- [80] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. 2025. Spargeattn: Accurate sparse attention accelerating any model inference. In *International Conference on Machine Learning (ICML)*.
- [81] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. 2024. LlamaTouch: A Faithful and Scalable Testbed for Mobile UI Task Automation. arXiv:2404.16054 [cs.HC] <https://arxiv.org/abs/2404.16054>
- [82] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. 2024. LlamaTouch: A Faithful and Scalable Testbed for Mobile UI Task Automation. arXiv:2404.16054 [cs.HC] <https://arxiv.org/abs/2404.16054>
- [83] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385 [cs.CL] <https://arxiv.org/abs/2401.02385>
- [84] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL] <https://arxiv.org/abs/2205.01068>
- [85] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhiyong Wang, and Beidi Chen. 2023. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048 [cs.LG] <https://arxiv.org/abs/2306.14048>
- [86] An Zou, Yuankai Xu, Yinchun Ni, Jintao Chen, Yehan Ma, Jing Li, Christopher Gill, Xuan Zhang, and Yier Jin. 2025. A Survey of Real-time Scheduling on Accelerator-based Heterogeneous Architecture for Time Critical Applications. arXiv:2505.11970 [cs.DC] <https://arxiv.org/abs/2505.11970>